



**SOLID** عبارة عن 5 مبادئ اتعملوا علشان يساعدوك تكتب كود:

. سهل يتقرا .

. سهل يتعدل .

. سهل تضيف عليه. Features.

. قليل الـ Bugs.

. أى Developer يشتغل بعدك يفهمه بسهولة .

تخيل إنك بنيت بيت

لو كل حاجة فى البيت متداخلة فى بعض، كل ما تيجى تغير باب، يقع الحيلة.

لكن لو البيت متقسم صح، تقدر تغير أى حاجة بسهولة.

وده بالضبط اللى SOLID بيعمله للكود.

## S = Single Responsibility Principle (SRP)

كل Class يكون مسئول عن حاجة واحدة فقط.

بمعنى...

كل Class يكون ليه سبب واحد فقط يخليه يتغير.

مثال غلط



```
public class User {  
  
    public void login() {  
  
    }  
  
    public void register() {  
  
    }  
  
    public void sendEmail() {  
  
    }  
  
    public void printReport() {  
  
    }  
  
    public void saveToDatabase() {  
  
    }  
  
}
```

الكلاس ده بيعمل كام حاجة؟

- Login, Register, Email
- Database
- Reports

يعنى خمس مسئوليات. وده مخالف لـ **SRP**.

الحل نفصل كل حاجة.

```
public class UserService {  
  
    public void login() {  
  
    }  
  
    public void register() {  
  
    }  
  
}
```

```
public class EmailService {  
  
    public void sendEmail() {  
  
    }  
  
}
```

```
public class UserRepository {  
  
    public void save(User user)  
  
    }  
  
}
```

```
public class ReportService {  
  
    public void printReport() {  
  
    }  
  
}
```

كل Class مسئول عن حاجة واحدة.

## O = Open/Closed Principle (OCP)

Software entities should be open for extension but closed for modification.

الكود يكون قابل لإضافة Features جديدة بدون ما تعدل فى الكود القديم.

Java code  بيحسب الخصومات.

بعد أسبوع .العميل قالك

ضيف feature

هتعمل إيه؟

```
if(type.equals("Platinum"))
```

```
if(type.equals("VIP"))
```

كل مرة بتعدل فى الكود القديم.

```
public class DiscountService {  
  
    public double calculate(String type, double price) {  
  
        if(type.equals("Regular"))  
            return price;  
  
        if(type.equals("Silver"))  
            return price * 0.9;  
  
        if(type.equals("Gold"))  
            return price * 0.8;  
  
        return price;  
    }  
}
```

```
public interface Discount {  
  
    double calculate(double price);  
  
}
```

الحل نعمل Interface.

```
public class RegularDiscount implements Discount {  
  
    @Override  
    public double calculate(double price) {  
        return price;  
    }  
}
```

```
public class SilverDiscount implements Discount {  
  
    @Override  
    public double calculate(double price) {  
        return price * 0.9;  
    }  
}
```

```
public class GoldDiscount implements Discount {  
  
    @Override  
    public double calculate(double price) {  
        return price * 0.8;  
    }  
}
```

العميل قال عايز VIP نعمل Class جديد فقط.



```
public class VIPDiscount implements Discount {  
  
    @Override  
    public double calculate(double price) {  
        return price * 0.7;  
    }  
}
```

## L = Liskov Substitution Principle (LSP)

Objects of a superclass should be replaceable with objects of a subclass without breaking the program.

أي Child Class لازم ينفع يحل مكان الـ Parent Class بدون ما البرنامج يبوظ.

```
class Car {  
  
    public void startEngine() {  
        System.out.println("Engine Started");  
    }  
  
    public void drive() {  
        System.out.println("Car is moving");  
    }  
  
}
```

أي عربية في الدنيا لازم:

- تدور
- تمشي

يبقى لو عملنا

```
class BMW extends Car {  
  
}
```

عندي

```
class Mercedes extends Car {  
  
}
```

دلوقتي

```
class Toyota extends Car {  
  
}
```

Method

```
public void testDrive(Car car) {  
  
    car.startEngine();  
    car.drive();  
  
}
```

أقدر أبعت

```
testDrive(new Toyota());
```

```
testDrive(new Mercedes());
```

```
testDrive(new BMW());
```

والبرنامج هيشغل. ليه؟ لأن كلهم عربيات. يبقى LSP متحقق.

## المشكلة

ظهر نوع جديد. Tesla

تسلا كهرباء. مفياش Engine أصلاً. لكن أنت ورثتها من Car.

```
class Tesla extends Car {  
  
    @Override  
    public void startEngine() {  
  
        throw new UnsupportedOperationException();  
  
    }  
  
}
```

`testDrive(new Tesla());`

دلوقتي

هياحصل. Exception.

يعنى أى مكان مستنى Car

لما أخذنا Tesla

البرنامج باظ. وده مخالف لـ LSP.

ليه التصميم غلط؟

لأننا افترضنا إن كل Car لازم يكون عندها Engine. وده مش حقيقى. تسلا عندها Motor.

الحل

نفصل المسئوليات.

نعمل

```
class Car {  
  
    public void drive() {  
  
        System.out.println("Driving");  
  
    }  
  
}
```

## ثم Interface

```
interface EngineCar {  
    void startEngine();  
}
```

```
class Tesla extends Car {  
    public void powerOn() {  
        System.out.println("Tesla Ready");  
    }  
}
```

```
class Mercedes extends Car implements EngineCar {  
    @Override  
    public void startEngine() {  
        System.out.println("Mercedes Engine Started");  
    }  
}
```

كدا منجبرش Tesla تعمل حاجة مش عندها.

```
class BMW extends Car implements EngineCar {  
    @Override  
    public void startEngine() {  
        System.out.println("BMW Engine Started");  
    }  
}
```

## I = Interface Segregation Principle (ISP)

Clients should not be forced to depend on interfaces they do not use.

متجبرش أي Class يطبق (Implement) Methods هو مش محتاجها.

اعمل Interfaces صغيرة ومتخصصة، بدل Interface واحدة ضخمة.

من الآخر "أنا ليه بستخدم حاجات مش محتاجها؟"

مثال Java غلط عندنا Interface

```
public interface Worker {  
  
    void work();  
  
    void eat();  
  
    void sleep();  
  
}
```

عندنا إنسان.

```
public class HumanWorker implements Worker {  
  
    @Override  
    public void work() {  
        System.out.println("Working");  
    }  
  
    @Override  
    public void eat() {  
        System.out.println("Eating");  
    }  
  
    @Override  
    public void sleep() {  
        System.out.println("Sleeping");  
    }  
  
}
```

```

public class RobotWorker implements Worker {

    @Override
    public void work() {

    }

    @Override
    public void eat() {

    }

    @Override
    public void sleep() {

    }

}

```

الشركة اشترت Robot.

المشكلة؟

الروبوت ياكل؟ ❌

بينام؟ ❌

يبقى هنكتب

```

@Override
public void eat() {

}

```

وده غلط.

ليه؟

لأن Interface فرضت عليه Methods مش محتاجها.

وده مخالف لـ ISP.



الحل

نقسم الـ Interface بدل واحدة كبيرة.

نعمل

```
public interface Sleepable {  
  
    void sleep();  
  
}
```

```
public interface Eatable {  
  
    void eat();  
  
}
```

```
public interface Workable {  
  
    void work();  
  
}
```

وكذا الإنسان

```
public class HumanWorker  
    implements Workable, Eatable, Sleepable {  
  
}
```

الروبوت

```
public class RobotWorker  
    implements Workable {  
  
}
```

الروبوت مش مجبر يعمل Eat أو Sleep وده ISP

## D = Dependency Inversion Principle (DIP)

أولاً يعني إيه **Dependency** ؟

**Dependency** معناها: Class يعتمد على Class تانى علشان يشتغل.

```
public class Car {  
  
    private Engine engine = new Engine();  
  
}
```

مين بيعتمد على مين؟

Car محتاج Engine.

يبقى Car depends on Engine

```
public class PetrolEngine {  
  
    public void start() {  
        System.out.println("Petrol Engine Started");  
    }  
  
}
```

المشكلة

تخيل عندك

ثم

كل حاجة تمام.

بعد سنة.

الشركة قالت: هنستخدم Diesel Engine.

هتعمل إيه؟ هتدخل تعدل فى Car.

بعد شهر. قالوا Electric Motor. تدخل

تعدل تانى.

يبقى المشكلة فين؟ Car عارف تفاصيل الـ Engine.

وده اسمه High Coupling يعنى ارتباط شديد.

```
public class Car {  
  
    private PetrolEngine engine = new PetrolEngine();  
  
    public void startCar() {  
  
        engine.start();  
  
    }  
  
}
```

**DIP** بيقول إيه؟

**High-level modules should not depend on low-level modules.  
Both should depend on abstractions.**

يعنى بالمصرى:

متخليش الكلاسات الكبيرة تعتمد على الكلاسات الصغيرة مباشرة. خليههم يعتمدوا على  
**Interface.**

الحل

نعمل Interface.

```
public interface Engine {  
  
    void start();  
  
}
```

Petrol

```
public class PetrolEngine implements Engine {  
  
    @Override  
    public void start() {  
  
        System.out.println("Petrol");  
  
    }  
  
}
```

Diesel

```
public class DieselEngine implements Engine {  
  
    @Override  
    public void start() {  
  
        System.out.println("Diesel");  
  
    }  
  
}
```

```

public class Car {

    private Engine engine;

    public Car(Engine engine) {

        this.engine = engine;
    }

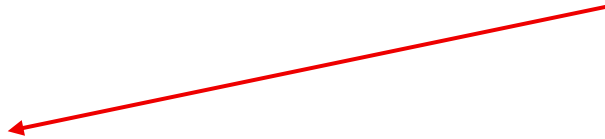
    public void startCar() {

        engine.start();
    }

}

```

دلوقتی Car



```

Engine engine = new PetrolEngine();

Car car = new Car(engine);

car.startCar();

```

دلوقتی أقدر أعمل



```

Engine engine = new DieselEngine();

Car car = new Car(engine);

```

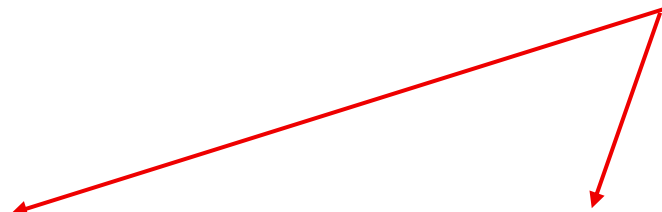
```

Engine engine = new ElectricEngine();

Car car = new Car(engine);

```

أو



هل عدلت فی Car ؟ لا. وده DIP.

Car مبقاش يعرف Petrol ولا Diesel.

يعرف Interface بس.